Scalable Transparent Arguments Of Knowledge

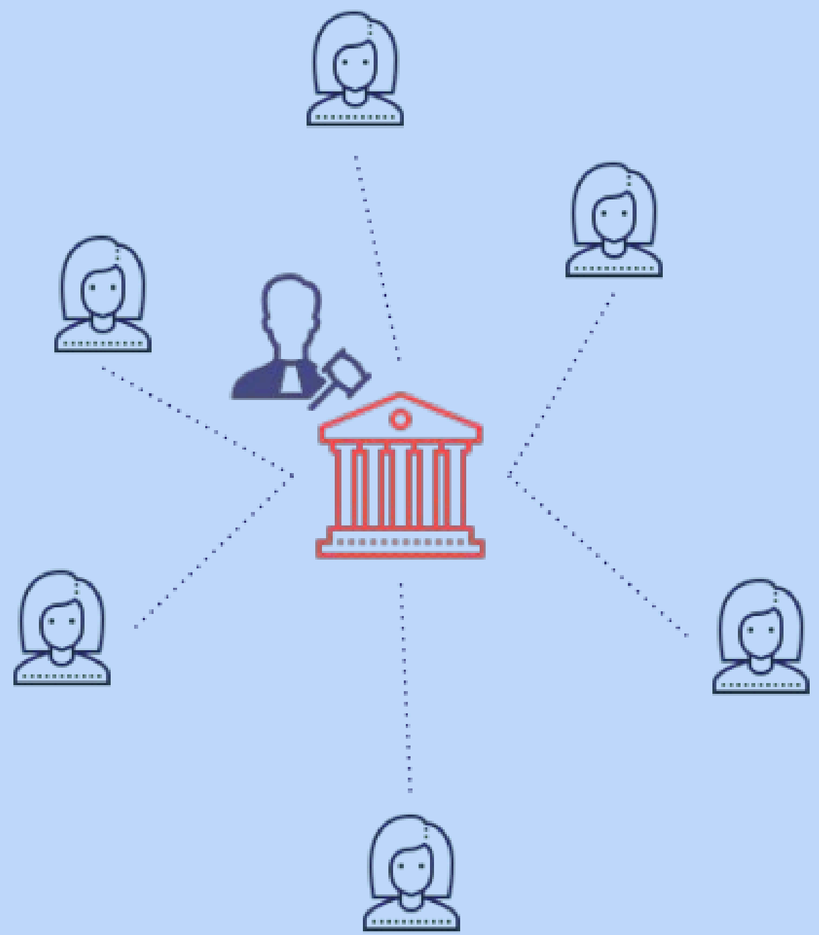
B Y

MUHAMMAD ZOHAIB MOHSIN

2026

Centralized

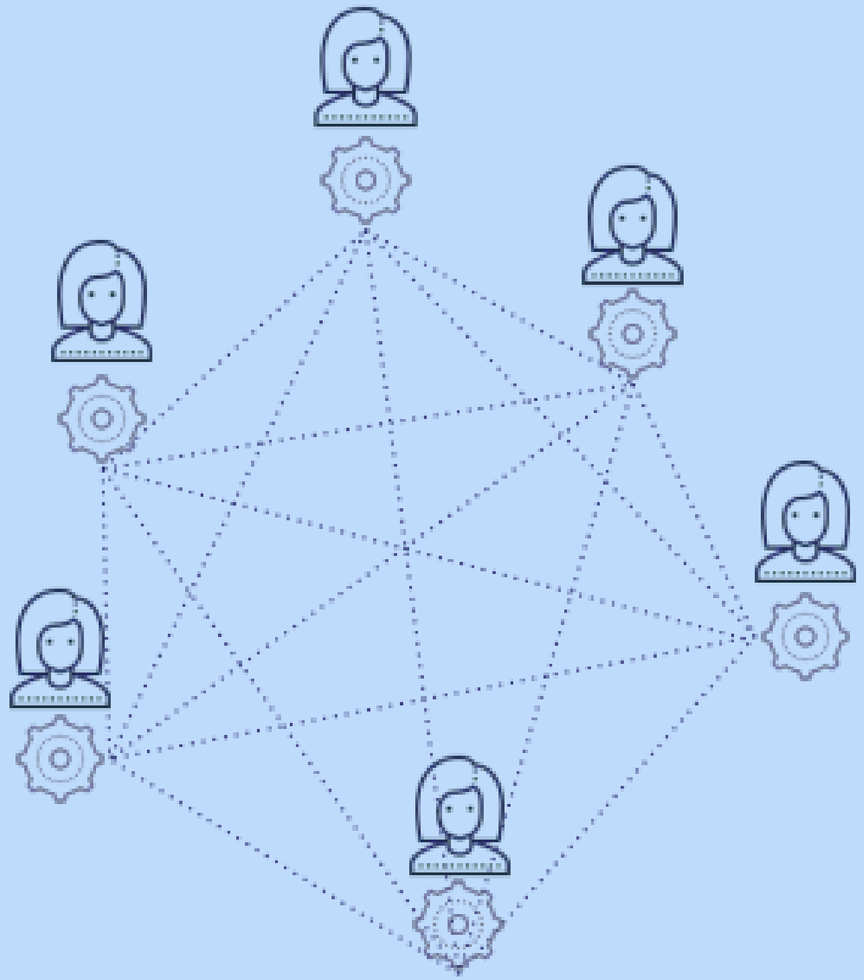
Banking System



Trust central party/auditor

Decentralized

Blockchain System

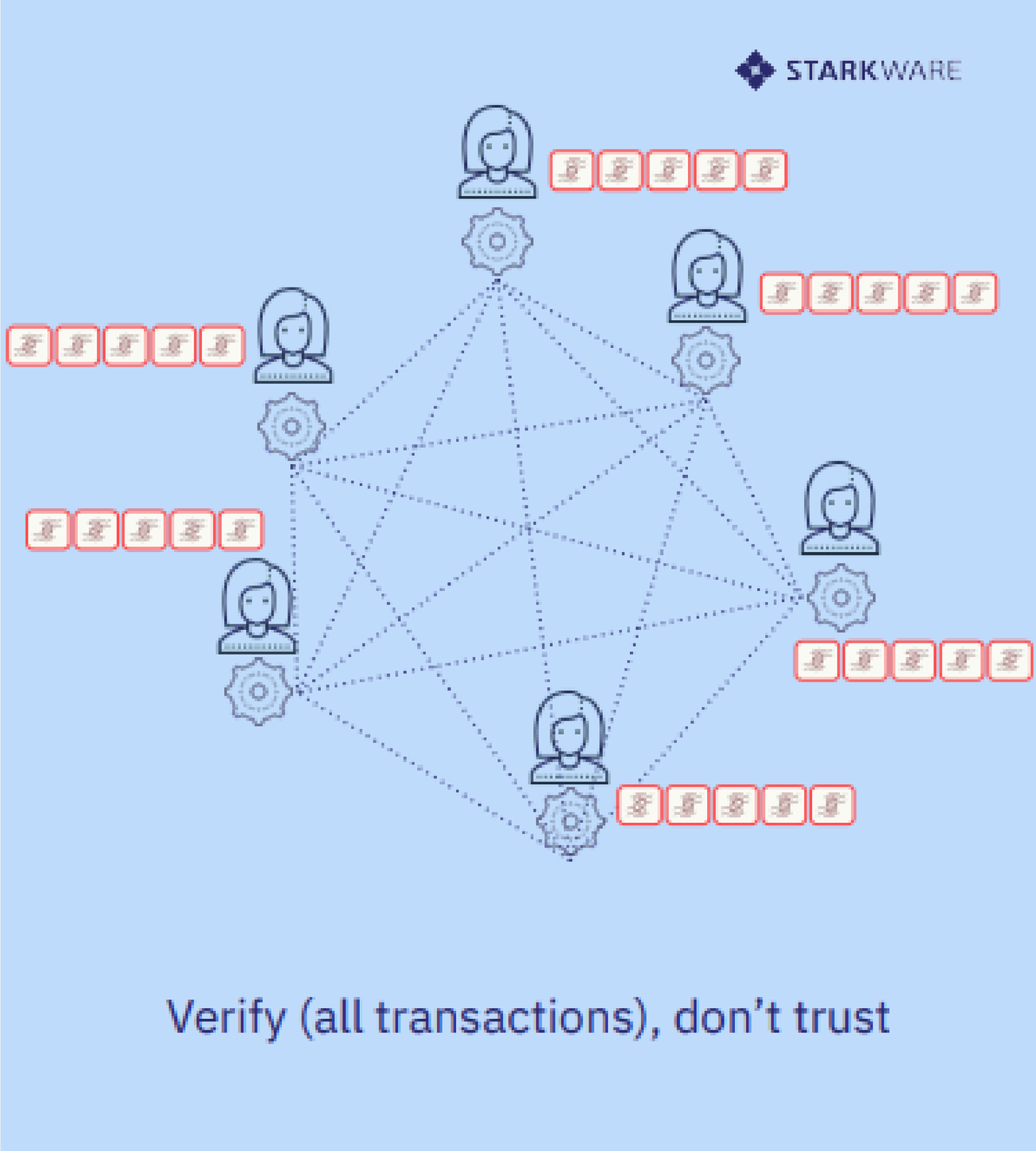


Verify (all transactions), don’t trust

Decentralized

Issues :

- Privacy
- Scalability



Goal :

The goal is to build some “**Integrity Mechanism**” like “**Empty Pot**” that should follow the following....

- Decentralized
- Privacy Protected
- Scalable

Smart Contracts :

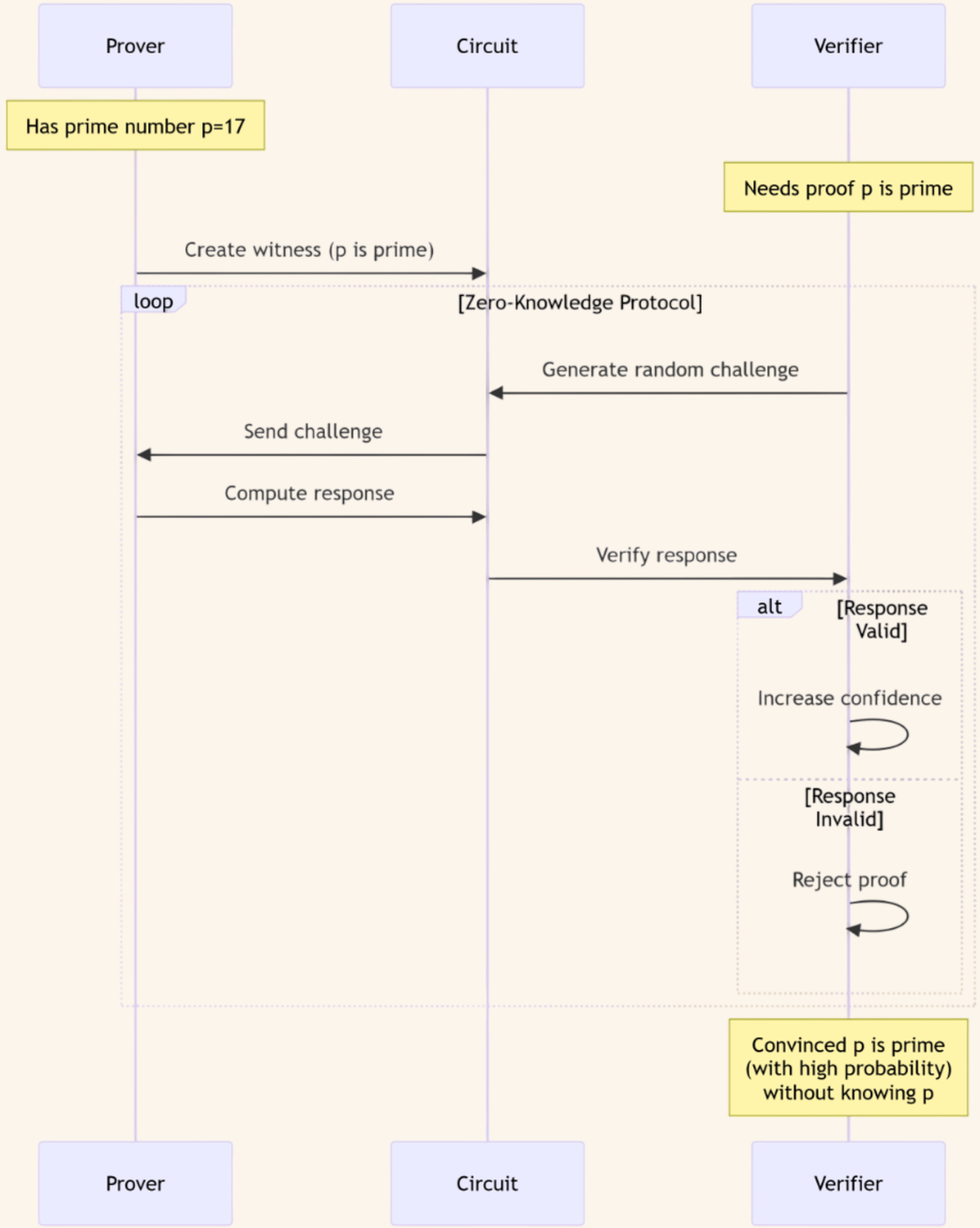
"Self-executing programs stored and run on a blockchain, automatically enforce and execute predefined rules or agreements without requiring intermediaries"

Zero Knowledge Proofs :

- Completeness
- Soundness
- Zero Knowledge

π = Prove (x,w)
Verify (π,x) accept/Reject

Zero-Knowledge Example



Zero-Knowledge Types

- Interactive Zero Knowledge Proofs
- Non-Interactive Zero Knowledge Proofs

We have studied the following approaches for the non-interactive zero-knowledge proofs

- zk-SNARK
- zk-STARK

Mathematical Foundations

Statement

Fibonacci Series of squares :

$a_{n+2} = a_{n+1}^2 + a_n^2$

“i Know a number **x** such that fibonacci series under mod some number **g** will result in the 1024th element = 123509834”

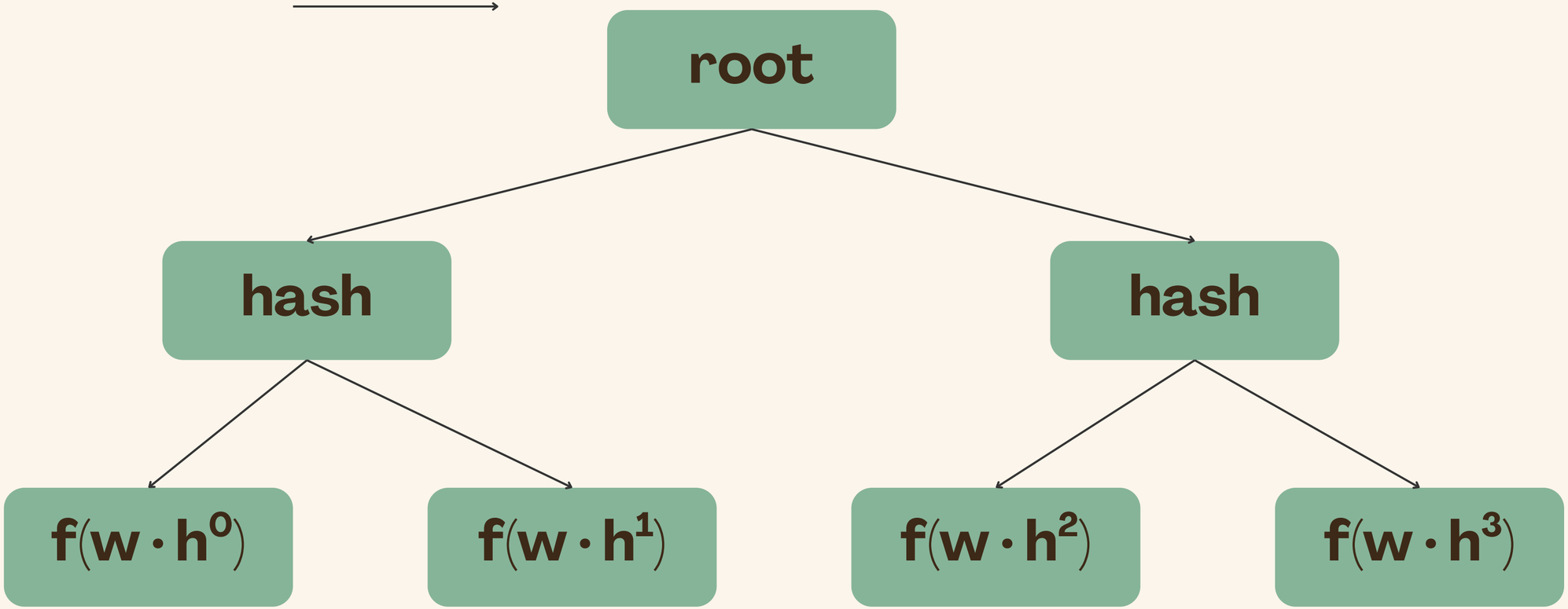
$a_{n+2} = a_{n+1} + a_n$ This equation is like

$a_{1024} = 12350983$ such that $a_0 = 1$ and $a_1 = x$

Lower Degree Extension

- Input
- Interpolate Input
- Extend over a larger domain

Commitment



Polynomial Contraints

$f(x) - 1 = 0$ for $x = 0$ ..... root : g^0

$f(x) - 12350983 = 0$ for $x = 1024$ .....root : g^{1024}

$f(g2x) - f(gx) - f(x) = 0$ for $x = g$, $0 <= x <= 1022$

$a_{n+2} = a_{n+1} + a_n$ This equation is like
$a_{1024} = 12350983$ such that $a_0 = 1$ and $a_1 = x$

FRI

"A function of the polynomial degree bounded by 1024 and the domain size is 8192 after performing one step of the fri it'll remain the half."

$CP = \alpha_0\, p_0(x) + \alpha_1\, p_1(x) + \alpha_2\, p_2(x)$ degree < 1024 domain = 8192

$CP_1 = \alpha_0\, p_0(x) + \alpha_1\, p_1(x) + \alpha_2\, p_2(x)$ degree < 512 domain = 4096

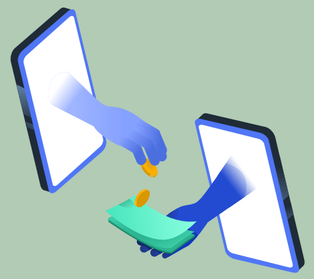
.

.

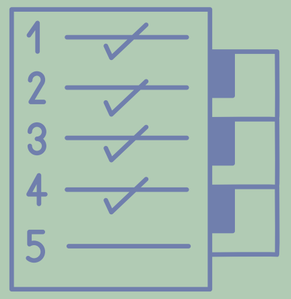
.

Scalability in Ethereum

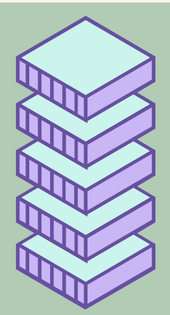
L2



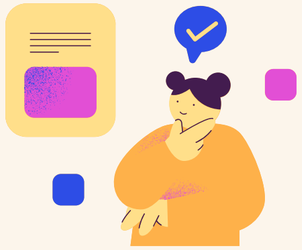
Transactions



Sequencer

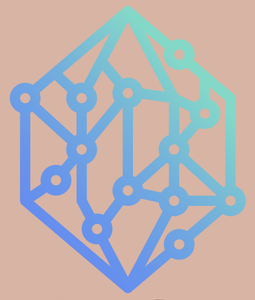


Block



Proof

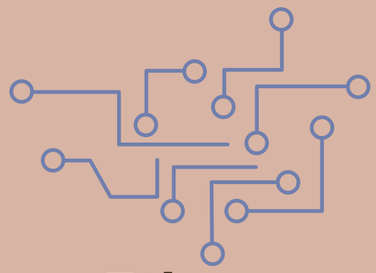
L1



Node



Verifier Contract

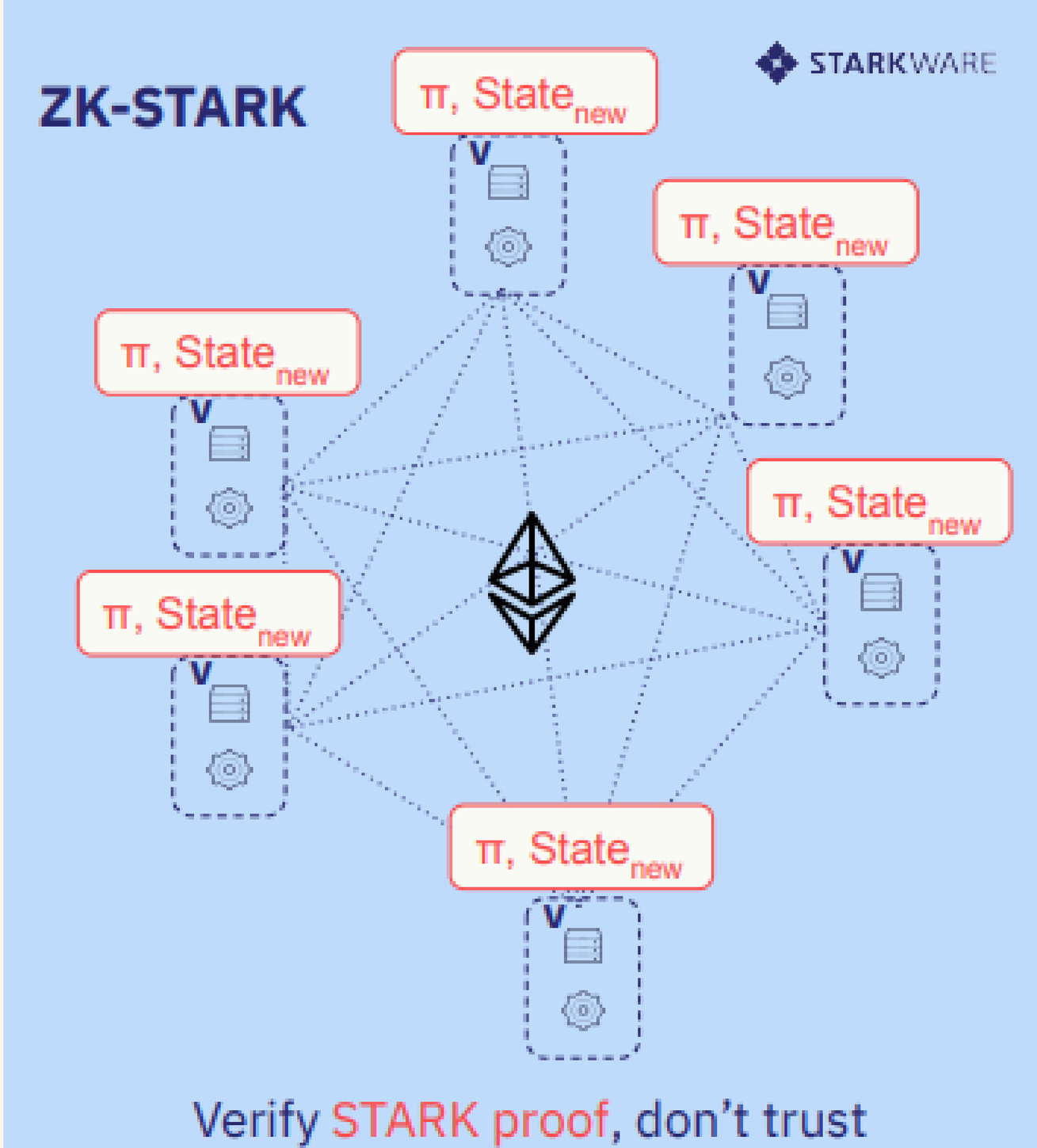


Ethereum

Decentralized

Issues :

- ~~Privacy~~
- ~~Scalability~~



Comparison

Feature	SNARK	STARK
Trusted Setup	Yes	No
Proof Size	Small	Larger
Scalability	Good But Limited	High
Transparency	Low	High
Post Quantum	Vulnerable	Secure

Cairo

“A programming language that is used to write the provable programs”

- Rust Inspired Structure
- Build Proofs
- Write Smart Contracts
- Cairo ⟶ Sierra ⟶ CASM

Cairo

“A programming language that is used to write the provable programs”

- Rust Inspired Structure
- Build Proofs
- Write Smart Contracts
- Cairo ⟶ Sierra ⟶ CASM

Cairo

Run the following command to get the scarb (a package manager for the cairo programs)

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.starkup.dev | sh
```

- scarb --version
- snforge --version
- sncast --version

Cairo

Run the following command to get the scarb (a package manager for the cairo programs)

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.starkup.dev | sh
```

- scarb --version
- snforge --version
- sncast --version

Cairo

Project Creation :

scarb new my_project

Project Compilation :

scarb build

Project Trace Generation :

scarb execute

Proof Generation :

scarb proof

Finally :

scarb verify

Cairo

START CODING



Remote Attestation of TPM Based Machines Using Starks

PROPOSAL

BY MUHAMMAD ZOHAIB MOHSIN

2026

Agenda

1. Preliminiaries
2. Introduction
3. Our Vision
4. Vision With Solution
5. Methodology
6. Limitations
7. Analysis
8. References
9. Future Work

PRELIMINARIES

Remote Attestation : Machine proves that it's running in a trusted, untampered state.

TPM : Trusted Plateform Module is basically a hardware chip act as hardware root of trust.

PCR's : Plateform Configuration Registers are used to find the states of the machine.

PCR's

PCR	Typical Contents	PCR	Typical Contents
0	BIOS/ Firmware	6	State Transitions
1	Plateform Config	7	Secure Boot Policy
2	Option ROM	8-9	OS Loader Specs.
3	Firmware Config	10	OS + Applications
4	Bootloader	11-15	Reserved OS use
5	Bootloader Config	16+	Dynamic PCR's

PRELIMINARIES

Endorsement Key : EK is a unique, permanent asymmetric key pair generated inside the TPM at manufacturing time.

Attestation Key : AIK is a special TPM-generated signing key used specifically for attestation purposes.

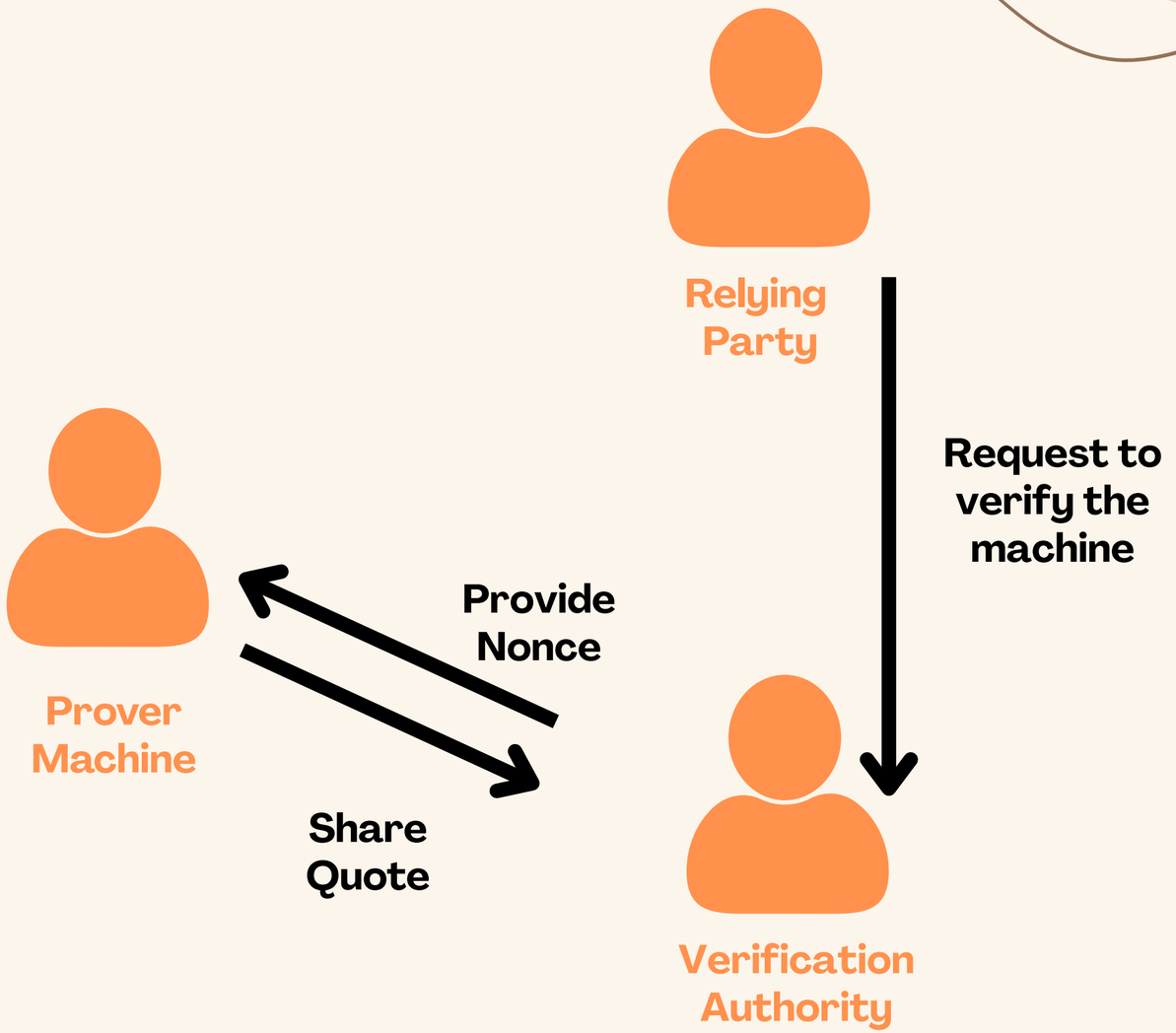
Quote : A TPM Quote is a digitally signed statement generated by the TPM.

Introduction

Current remote attestation schemes—like those used by Google Cloud, Intel SGX, AMD SEV—typically rely on:

- Hardware vendors (Intel/AMD) to issue endorsements.
- Cloud providers (Google, AWS) to perform or forward attestation.
- External verifiers needing to trust certificates, metadata APIs, or cloud services.

Traditional Flow



“Verification Authority Provide some certificate to the prover or issue some token”

Issues

- Totally Centralized
- Hardware cert chains, CA’s chains.
- Sharing of RAW logs.

Our Vision

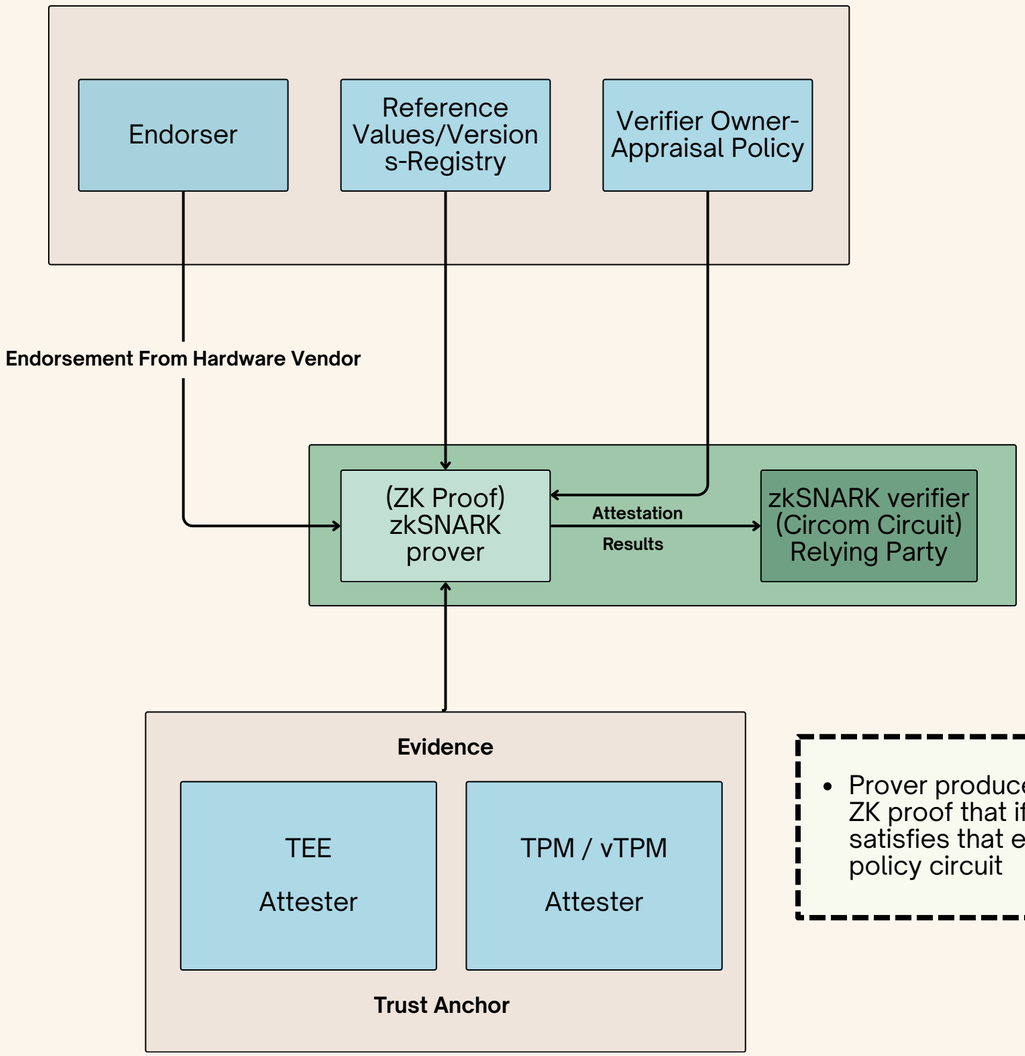
Trust Anchors Decentralization
Open Verification Eco system
Privacy Preserving
Cryptographic Verifiability & Transparency

Vision with Solution

Vision	Solution
Trust Anchors Decentralization Open Verification Eco system	Smart Contracts
Privacy Preserving Cryptographic Verifiability & Transparency	Zero Knowledge Proofs

Approach 1-SNARK

System Design



- Prover produces a ZK proof that if satisfies that exact policy circuit

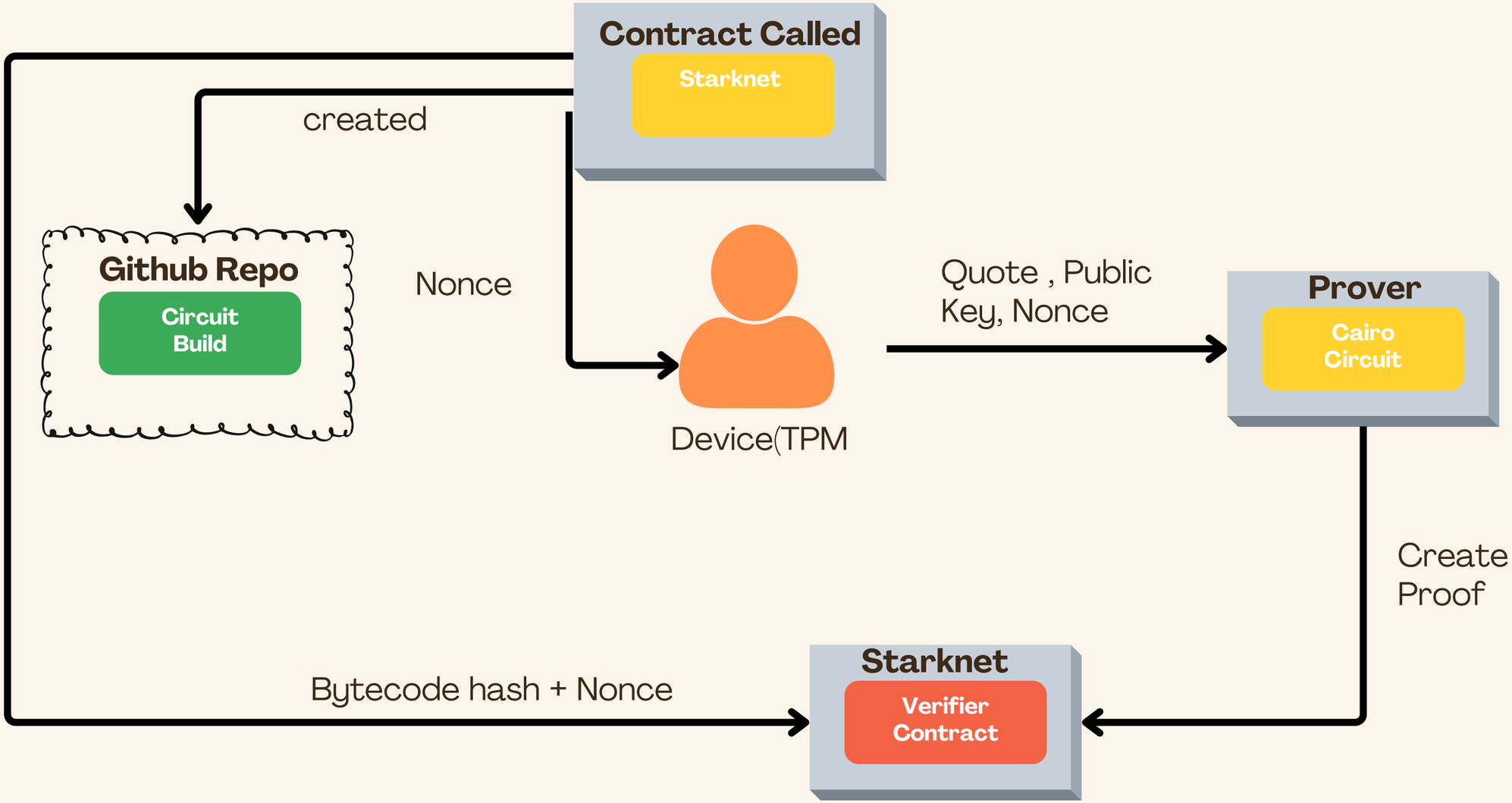
Relying party defines what constitutes “trust” via :

- A circom circuit encoding the appraisal logic
- A public verification key representing that policy version

The circuit enforces:

- What vendor root keys are trusted
- Which firmware versions are acceptable
- How measurement hashes are checked
- Optional Additional rules (e.g, deta sealing, domain binding)

APPROACH 2- STARK



Circuit Properties

- Quote signature bindings(Digest Binding)
- Ak public key hash
- Nonce freshness
- Pcr’s membership vs Golden list

Mandatory Checks

Check	Role
Nonce	To check freshness
Sign	To validate a real TPM
Digest Binding	To cross verify states
Golden List	To implant the requirements

Possible Attacks

Attack	Defense
Submit Old Proof	Check nonce freshness
Submit Other Machine logs	Validate a real TPM sgin
Mark one/more pcr’s as member of the golden list	Cross verify states via digest binding

Flow

Contract: TPMAttestationVerifier.cairo stores program_hash, golden_list_hash, nonce logic and verify_attestation().

Circuit: attestation_circuit.cairo implements public inputs and the 4 checks.

Scripts: local_scripts/generate_tpm_evidence.sh and submit_attestation.sh automate evidence creation and submission.

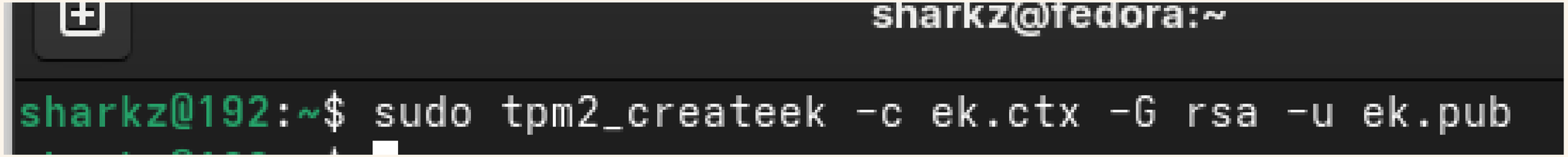
Step 1 — Read PCR Values

Commad: sudo tpm2_pcrread

```
sharkz@192:~$ sudo tpm2_pcrread
  sha1:
    0 : 0xC3C0C070C09CFE123519ECA7FD2C907A43601343
    1 : 0x39633BC9AD39786BF7F706EDF81F87BD76F0E161
    2 : 0xB2A83B0EBF2F8374299A5B2BDFC31EA955AD7236
    3 : 0xB2A83B0EBF2F8374299A5B2BDFC31EA955AD7236
    4 : 0xE0232106A76FA8C06A8EC76890BD53F629DFC27D
    5 : 0x8E291042E3DDDE893EB7F618D8219A1DA8FBE63A
    6 : 0xB2A83B0EBF2F8374299A5B2BDFC31EA955AD7236
    7 : 0x293C1DEF1D158EAD0D85D9D12DD00FE69175FB36
    8 : 0x21F26A0DCF284C5997FDDFA2F07E8195FEF62F7C
    9 : 0x35D777BADA5C47ABF08A5B555C780A03AD4B17CC
   10: 0xED96566EAD6703E8D1E3A4E8489545CAD7D40003
```

Step 2 — Create Endorsement Key

Commad: sudo tpm2_createek -c ek.ctx -G rsa -u ek.pub



Step 3 — Create Attestation Key

Commad: sudo tpm2_createak -C ek.ctx -c ak.ctx -G rsa -g sha256 -s rsassa -u ak.pub -n ak.name

```
sharkz@192:~$ sudo tpm2_createak -C ek.ctx -c ak.ctx -G rsa -g sha256 -s rsassa
-u ak.pub -n ak.name
loaded-key:
  name: 000bdcacec87e8dced493c6599c7638386f70ffb57d4f1ac60406f2545c57f2ba585
  qualified name: 000bcd854d12569dcb4b19dfd8daddf47195f0965eeeb5de79a265382f94ac
2e89e2
sharkz@192:~$
```

Step 4 — Generate TPM Quote

Commad: sudo tpm2_quote -c ak.ctx -l sha256:0,1,2,3,4,5,6,7 -q 6e6f6e6365313233 -m quote.msg -s quote.sig -o quote.pcr



If you run tpm2_quote again but forget to regenerate quote.sig and quote.pcr, the signature will

Limitations

TPM Based

Well Both the TPM and TEE are used for the confidential computing and their role is to provide system integrity. This solution is only for the tPM devices

Linux Dependent

Linly Linux provide the quote directly while other operating systems doesn’t allow e.g windows

Hardware Manufacturer

We tried to remove every kind of dependency but we dependent on hardware manufacturer for the final trustworthy values

Analysis

Strengths

- Strong privacy and Verifiable computation
- Portable across all TEEs and clouds
- Reduced attack surface and Trust minimization

Weakness

- High engineering complexity
- Larger ZK proof generation time
- No existing industry standard

Related Work

zk-Based

- Remote Attestation using zero knowledge proof has already been discussed
- It’s highly dependent on pre shared keys
- No way to share machine logs
- Verbal Attestation being done
- SNARK-Based only

Traditional

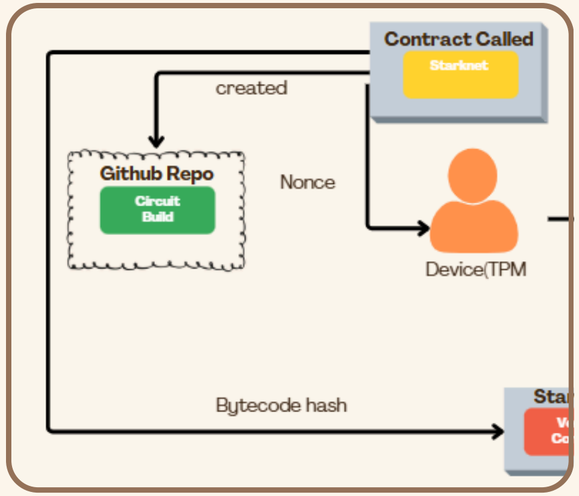
- Direct sharing of the logs
- Dependency on the CA’s.
- Process being by almost all the remote attestation authorities

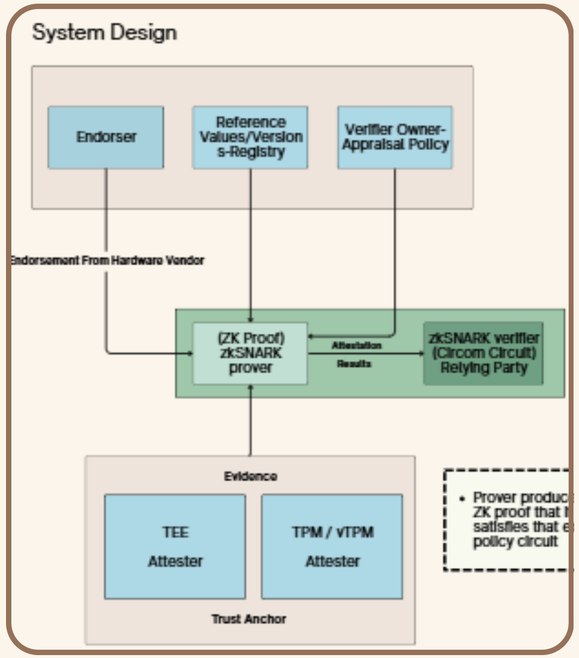
References

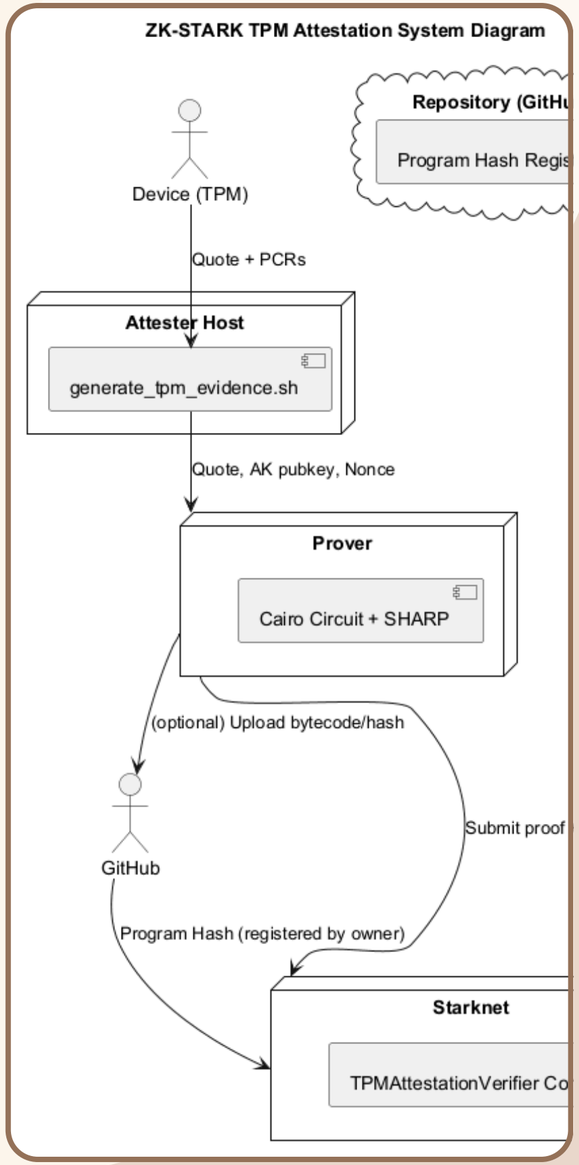
- Ebrahimi, S., & Hassanizadeh, P. (2024). From Interaction to Independence: zkSNARKs for Transparent and Non-Interactive Remote Attestation. NDSS 2024. https://eprint.iacr.org/2024/1068
- StarkWare. Cairo Programming Language Documentation. https://www.cairo-lang.org/docs
- Wikipedia. Zero-Knowledge Proof. Online; accessed 2026. https://en.wikipedia.org/wiki/Zero-knowledge_proof
- Trusted Computing Group (TCG). TPM 2.0 Library Specification. Revision 1.59, 2020. https://trustedcomputinggroup.org/tpm-library-specification
- Intel. Intel Software Guard Extensions (SGX) Developer Guide. 2022. https://www.intel.com/content/www/us/en/develop/documentation
- Backes, M., Hanzlik, L., Herzberg, A., Kate, A., & Pryvalov, I. (2019). Efficient Non-Interactive Zero-Knowledge Proofs in Cross-Domains without Trusted Setup. Cryptology ePrint Archive, Paper 2019/063. https://eprint.iacr.org/2019/063
- Wikipedia. Trusted Platform Module. Online; accessed 2026. https://en.wikipedia.org/wiki/Trusted_Platform_Module

Future Work

- Zero Knowledge based attestation can be done of TEE Machine.
- Batch attestation can be also taken under consideration
- Continuous attestation can be considered.
- Making it efficient for the users with other operating systems as well







Thank You!

By Muhammad Zohaib Mohsin

Mscs 24031